Dylan Thornburg
ECEN 511 Programming Assignment 2

Caching Simulator

Understanding the Task

I need to implement a cache simulator that simulates caching for reads/writes, multiple different associativities, multiple different replacement policies, as well as configurations. For this sim, I will only use the default configuration regarding addr width, line size, mem size, and cache size to make things easier. Using these, we have:

Addr width=16 bits
Cache size = 2^10=1024 bytes
Line size = 2^4=16 bytes
Mem size = 2^20=1048576 bytes (only used for initialization)

From (cache size / line size), we can get the total number of lines which is 64 and log2(64)=6. This gives us 6 index bits, and from line size, we get 4 offset bits. Because addr is 16 bits, the remaining 6 bits go to tag.

Implementation

The total number of cache lines is derived from the cache and line size parameters (like above), and the number of sets is computed from the associativity, with direct-mapped caches treated as one-way set-associative caches and fully associative caches implemented as a single set containing all lines. Tag, index, and offset are computed dynamically from the address width, number of sets, and line size. Each cache line maintains a valid bit, tag, and data block, allowing all associativities to be simulated using the same access logic.

Since we don’t actually have to write to memory, reads and writes treated identically (inside write, we call read). On each access, the target set is searched for a matching valid tag for a hit. On a miss, an invalid line is used when available, resulting in a compulsory miss. If this fails, a line is evicted and replaced. Random replacement selects a line uniformly at random (srand), LRU evicts the least recently accessed line using timestamp-based tracking, and optimal replacement evicts the block whose next use occurs farthest in the future. Conflict vs capacity misses are differentiated based on whether our cold misses = number of lines. If cold misses < number of lines, it's a conflict. If it's = number of lines, then we know we've maxed out our cache and therefore we are at a capacity limit from here on.

Direct-Mapped Simulations

I will start with a smaller set of addresses to test (there will be no capacity misses here)

order	Addr (hex)	Tag 6-bits	Index 6-bits	Offset 4-bits	Hit/Miss?

1	0000	000000	000000	0000	Miss (com)
2	0001	000000	000000	0001	hit
3	0040	000000	000100	0000	Miss (com)
4	1040	000100	000100	0000	Miss & R (conf)
5	8080	100000	001000	0000	Miss (com)
6	8081	100000	001000	0001	hit
7	8080	100000	001000	0000	hit
8	1040	000100	000100	0000	hit

So from this, expect:
4 hits
3 compulsory misses
1 conflict miss
0 capacity misses
50% hit rate

When running in Sim we see:

```
cache size: 1024 Bytes
associativity: direct-mapped
replacement policy: random
trace #1: R 0x0000
READ: Address = 0x0
trace #2: R 0x0001
READ: Address = 0x1
trace #3: R 0x0040
READ: Address = 0x40
trace #4: R 0x1040
READ: Address = 0x1040
trace #5: R 0x8080
READ: Address = 0x8080
trace #6: R 0x8081
READ: Address = 0x8081
trace #7: R 0x8080
READ: Address = 0x8080
trace #8: R 0x1040
READ: Address = 0x1040
================ Cache Statistics  =====================
for 8 accesses ...
hits: 4
compulsory misses: 3
capacity misses: 0
conflict misses: 1
cache hit rate: 50.0%

Process returned 0 (0x0)   execution time : 0.021 s
Press any key to continue.
```

So our implementation works at least for direct-mapped. Note that optimal vs lru vs random doesn't matter here as there is only one index to ever replace during a conflict miss.

Now simulating for the Trace that was given, these are the results:

```
================ Cache Statistics  =====================
for 1000 accesses ...
hits: 12
compulsory misses: 64
capacity misses: 740
conflict misses: 184
cache hit rate: 1.2%

Process returned 0 (0x0)   execution time : 0.121 s
Press any key to continue.
```

To prove LRU and Optimal doesn't change the results, here are those simulations:

LRU:

```
================ Cache Statistics  =====================
for 1000 accesses ...
hits: 12
compulsory misses: 64
capacity misses: 740
conflict misses: 184
cache hit rate: 1.2%

Process returned 0 (0x0)   execution time : 0.106 s
Press any key to continue.
```

Optimal:

```
================ Cache Statistics  =====================
for 1000 accesses ...
hits: 12
compulsory misses: 64
capacity misses: 740
conflict misses: 184
cache hit rate: 1.2%

Process returned 0 (0x0)   execution time : 0.108 s
Press any key to continue.
```

2, 4, and 8 Way Set Associative Cache Testbench Simulations

Before I begin the main code sims, I will complete more short, preliminary testbenches for each way and each policy.

2-Way

2-Way Random:

order	Addr (hex)	Tag 7-bits	Index 5-bits	Offset 4-bits	Hit/Miss?
1	02A3	0000 001	0 1010	0011	Comp Miss
2	06A0	0000 011	0 1010	0000	Comp miss
3	06A3	0000 011	0 1010	0011	hit
4	0EA3	0000 111	0 1010	0011	Miss & R (conf)
5	1283	0001 001	0 1000	0011	Comp miss
6	168A	0001 011	0 1000	1010	Comp miss
7	1A88	0001 101	0 1000	1000	Miss & R (conf)
8	1283	0001 001	0 1000	0011	Could be hit or Miss depending on whats replaced

Expected Results:
4 cold misses
2 conf misses and 2 hits (hitrate =25%) if order 6 is replaced. 3 conf misses and 1 hit (hitrate=12.5%) if order 5 is replaced.

Results:

```
================= Cache Statistics  =======================
for 8 accesses ...
hits: 2
compulsory misses: 4
capacity misses: 0
conflict misses: 2
cache hit rate: 25.0%

Process returned 0 (0x0)   execution time : 0.016 s
Press any key to continue.
```

AND

```
================= Cache Statistics  =======================
for 8 accesses ...
hits: 1
compulsory misses: 4
capacity misses: 0
conflict misses: 3
cache hit rate: 12.5%

Process returned 0 (0x0)   execution time : 0.009 s
Press any key to continue.
```

These results match exactly what is expected.

2-Way LRU:

order	Addr (hex)	Tag 7-bits	Index 5-bits	Offset 4-bits	Hit/Miss?
1	02A3	0000 001	0 1010	0011	Comp Miss
2	06A0	0000 011	0 1010	0000	Comp miss
3	06A3	0000 011	0 1010	0011	hit
4	0EA3	0000 111	0 1010	0011	Miss & R (conf)
5	1283	0001 001	0 1000	0011	Comp miss
6	168A	0001 011	0 1000	1010	Comp miss

7	1A88	0001 101	0 1000	1000	Miss & R (conf)
8	1283	0001 001	0 1000	0011	Miss & R (conf)

Expected Results:
4 cold misses
3 conf misses and 1 hit (hitrate=12.5%)

Results

```
================= Cache Statistics  =======================
for 8 accesses ...
hits: 1
compulsory misses: 4
capacity misses: 0
conflict misses: 3
cache hit rate: 12.5%

Process returned 0 (0x0)   execution time : 0.011 s
Press any key to continue.
```

Works as expected

2-Way Optimal:

order	Addr (hex)	Tag 7-bits	Index 5-bits	Offset 4-bits	Hit/Miss?
1	02A3	0000 001	0 1010	0011	Comp Miss
2	06A0	0000 011	0 1010	0000	Comp miss
3	06A3	0000 011	0 1010	0011	hit
4	0EA3	0000 111	0 1010	0011	Miss & R (conf)
5	1283	0001 001	0 1000	0011	Comp miss

6	168A	0001 011	0 1000	1010	Comp miss
7	1A88	0001 101	0 1000	1000	Miss & R (conf)
8	1283	0001 001	0 1000	0011	hit

Expected Results:
4 cold misses
2 conf misses and 2 hit (hitrate=25%) because optimal can see future and always knows where to best replace data.

Results:

```
================= Cache Statistics  =======================
for 8 accesses ...
hits: 2
compulsory misses: 4
capacity misses: 0
conflict misses: 2
cache hit rate: 25.0%

Process returned 0 (0x0)   execution time : 0.010 s
Press any key to continue.
```

This confirms what is expected.

4-Way

4-Way Random, LRU, & optimal:

order	Addr (hex)	Tag 8-bits	Index 4-bits	Offset 4-bits	Hit/Miss?
1	02A3	0000 0010	1010	0011	Comp miss
2	06A0	0000 0110	1010	0000	Comp miss
3	06A3	0000 0110	1010	0011	hit
4	0EA3	0000 1110	1010	0011	Comp miss

5	1283	0001 0010	1000	0011	Comp miss
6	168A	0001 0110	1000	1010	Comp miss
7	1A88	0001 1010	1000	1000	Comp miss
8	1283	0001 0010	1000	0011	hit

Expected Results:
6 cold misses
2 hits (hitrate =25%)
Note that for this config, we expect Random to match LRU to match optimal because no replacement happens here.

Results:

Random

```
================ Cache Statistics  =======================
for 8 accesses ...
hits: 2
compulsory misses: 6
capacity misses: 0
conflict misses: 0
cache hit rate: 25.0%

Process returned 0 (0x0)   execution time : 0.016 s
Press any key to continue.
```

LRU

```
================ Cache Statistics  =======================
for 8 accesses ...
hits: 2
compulsory misses: 6
capacity misses: 0
conflict misses: 0
cache hit rate: 25.0%

Process returned 0 (0x0)   execution time : 0.017 s
Press any key to continue.
```

Optimal

```
================= Cache Statistics  =======================
for 8 accesses ...
hits: 2
compulsory misses: 6
capacity misses: 0
conflict misses: 0
cache hit rate: 25.0%

Process returned 0 (0x0)   execution time : 0.018 s
Press any key to continue.
```

As we can see, all results match what is expected.

8-Way

8-Way Random, LRU, & optimal:

order	Addr (hex)	Tag 8-bits	Index 4-bits	Offset 4-bits	Hit/Miss?
1	02A3	0000 0010 1	010	0011	Comp miss
2	06A0	0000 0110 1	010	0000	Comp miss
3	06A3	0000 0110 1	010	0011	hit
4	0EA3	0000 1110 1	010	0011	Comp miss
5	1283	0001 0010 1	000	0011	Comp miss
6	168A	0001 0110 1	000	1010	Comp miss
7	1A88	0001 1010 1	000	1000	Comp miss
8	1283	0001 0010 1	000	0011	hit

Expected Results:
6 cold misses
2 hits (hitrate =25%)
Note that for this config, like with the 4-way, we expect Random to match LRU to match optimal because no replacement happens here.

Results:

Random

```
================= Cache Statistics  =======================
for 8 accesses ...
hits: 2
compulsory misses: 6
capacity misses: 0
conflict misses: 0
cache hit rate: 25.0%

Process returned 0 (0x0)   execution time : 0.011 s
Press any key to continue.
```

LRU

```
================= Cache Statistics  =======================
for 8 accesses ...
hits: 2
compulsory misses: 6
capacity misses: 0
conflict misses: 0
cache hit rate: 25.0%

Process returned 0 (0x0)   execution time : 0.017 s
Press any key to continue.
```

Optimal

```
================= Cache Statistics  =======================
for 8 accesses ...
hits: 2
compulsory misses: 6
capacity misses: 0
conflict misses: 0
cache hit rate: 25.0%

Process returned 0 (0x0)   execution time : 0.010 s
Press any key to continue.
```

Again just as expected.

2, 4, and 8 Way Set Associative Cache Simulations For Given Trace

Now I will test for the given trace.

2-Way

Random (multiple b/c random can change the results)

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 16
compulsory misses: 64
capacity misses: 737
conflict misses: 183
cache hit rate: 1.6%

Process returned 0 (0x0)   execution time : 0.097 s
Press any key to continue.
```

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 13
compulsory misses: 64
capacity misses: 740
conflict misses: 183
cache hit rate: 1.3%

Process returned 0 (0x0)   execution time : 0.100 s
Press any key to continue.
```

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 18
compulsory misses: 64
capacity misses: 736
conflict misses: 182
cache hit rate: 1.8%

Process returned 0 (0x0)   execution time : 0.098 s
Press any key to continue.
```

LRU

```
================ Cache Statistics  =======================
for 1000 accesses ...
hits: 17
compulsory misses: 64
capacity misses: 736
conflict misses: 183
cache hit rate: 1.7%

Process returned 0 (0x0)   execution time : 0.104 s
Press any key to continue.
```

Optimal

```
================ Cache Statistics  =======================
for 1000 accesses ...
hits: 56
compulsory misses: 64
capacity misses: 699
conflict misses: 181
cache hit rate: 5.6%

Process returned 0 (0x0)   execution time : 0.125 s
Press any key to continue.
```

4-Way

Random

```
================ Cache Statistics  =======================
for 1000 accesses ...
hits: 16
compulsory misses: 64
capacity misses: 884
conflict misses: 36
cache hit rate: 1.6%

Process returned 0 (0x0)   execution time : 0.100 s
Press any key to continue.
```

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 10
compulsory misses: 64
capacity misses: 889
conflict misses: 37
cache hit rate: 1.0%

Process returned 0 (0x0)   execution time : 0.097 s
Press any key to continue.
```

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 11
compulsory misses: 64
capacity misses: 889
conflict misses: 36
cache hit rate: 1.1%

Process returned 0 (0x0)   execution time : 0.101 s
Press any key to continue.
```

LRU

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 11
compulsory misses: 64
capacity misses: 888
conflict misses: 37
cache hit rate: 1.1%

Process returned 0 (0x0)   execution time : 0.100 s
Press any key to continue.
```

Optimal

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 86
compulsory misses: 64
capacity misses: 814
conflict misses: 36
cache hit rate: 8.6%

Process returned 0 (0x0)   execution time : 0.114 s
Press any key to continue.
```

8-Way

Random

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 13
compulsory misses: 64
capacity misses: 893
conflict misses: 30
cache hit rate: 1.3%

Process returned 0 (0x0)   execution time : 0.107 s
Press any key to continue.
```

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 10
compulsory misses: 64
capacity misses: 896
conflict misses: 30
cache hit rate: 1.0%

Process returned 0 (0x0)   execution time : 0.101 s
Press any key to continue.
```

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 8
compulsory misses: 64
capacity misses: 898
conflict misses: 30
cache hit rate: 0.8%

Process returned 0 (0x0)   execution time : 0.103 s
Press any key to continue.
```

LRU

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 7
compulsory misses: 64
capacity misses: 898
conflict misses: 31
cache hit rate: 0.7%

Process returned 0 (0x0)   execution time : 0.104 s
Press any key to continue.
```

Optimal

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 97
compulsory misses: 64
capacity misses: 809
conflict misses: 30
cache hit rate: 9.7%

Process returned 0 (0x0)   execution time : 0.105 s
Press any key to continue.
```

Overall the results show that for this set of accesses, the two set-associative cache works best for random and LRU. Ordinarily, we would expect higher associativity to have a higher hit rate, but this sequence must not have abused temporal locality as much. We can still see the conflict misses go down quite a bit, which is expected.

Note that optimal always performed the best and got better for higher associativity, which is expected. Full associativity + optimal is the best/ideal cache config, but it is hypothetical (cant see the future in reality).

Fully Associative Cache Testbench Simulations

Now I will test fully associative, which I've described as associativity =0 in my code, but because I have 64 lines, if I set associativity to 64, that would also work. For these simulations, there should never be any conflict misses.

Fully Assoc

Random, LRU, & Optimal (no replacing, so all should match):

order	Addr (hex)	Tag 12-bits	Offset 4-bits	Hit/Miss?
1	02A3	0000 0010 1010	0011	Comp Miss
2	06A0	0000 0110 1010	0000	Comp miss
3	06A3	0000 0110 1010	0011	hit
4	0EA3	0000 1110 1010	0011	Comp miss
5	1283	0001 0010 1000	0011	Comp miss

6	168A	0001 0110 1000	1010	Comp miss
7	1A88	0001 1010 1000	1000	Comp miss
8	1283	0001 0010 1000	0011	Hit

Expected for all policies:
Hits:2
Comp miss:6
Cap miss:0
Conf miss:0
Hitrate =25%

Results:

Random

```
================ Cache Statistics  =======================
for 8 accesses ...
hits: 2
compulsory misses: 6
capacity misses: 0
conflict misses: 0
cache hit rate: 25.0%

Process returned 0 (0x0)   execution time : 0.017 s
Press any key to continue.
```

LRU

```
================ Cache Statistics  =======================
for 8 accesses ...
hits: 2
compulsory misses: 6
capacity misses: 0
conflict misses: 0
cache hit rate: 25.0%

Process returned 0 (0x0)   execution time : 0.017 s
Press any key to continue.
```

Optimal

```
================= Cache Statistics  =======================
for 8 accesses ...
hits: 2
compulsory misses: 6
capacity misses: 0
conflict misses: 0
cache hit rate: 25.0%

Process returned 0 (0x0)   execution time : 0.010 s
Press any key to continue.
```

Fully Associative Cache Simulations For Given Trace

Random (multiple because its random)

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 14
compulsory misses: 64
capacity misses: 922
conflict misses: 0
cache hit rate: 1.4%

Process returned 0 (0x0)   execution time : 0.112 s
Press any key to continue.
```

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 12
compulsory misses: 64
capacity misses: 924
conflict misses: 0
cache hit rate: 1.2%

Process returned 0 (0x0)   execution time : 0.100 s
Press any key to continue.
```

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 11
compulsory misses: 64
capacity misses: 925
conflict misses: 0
cache hit rate: 1.1%

Process returned 0 (0x0)   execution time : 0.101 s
Press any key to continue.
```

LRU

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 11
compulsory misses: 64
capacity misses: 925
conflict misses: 0
cache hit rate: 1.1%

Process returned 0 (0x0)   execution time : 0.107 s
Press any key to continue.
```

Optimal

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 111
compulsory misses: 64
capacity misses: 825
conflict misses: 0
cache hit rate: 11.1%

Process returned 0 (0x0)   execution time : 0.115 s
Press any key to continue.
```

Overall we can see that optimal as expected performed the best by far. We also see that same behavior as with the 2,4,8 set associativity where the higher the associativity, the worse LRU performs for this given access list. It's not really what we expect, but still possible, and shows that LRU is not always better than random (even if it is most of the time). We can also see there were no conflict misses ever, which is expected due to the way full associativity works.

Experiment: Change Config so Cache Never Has a Capacity Problem

Changing the default config file to have an addr that's 20 bits long and cache size that's 14 bits long means we now have 1024 lines. This ensures that for our list of 1000 accesses, we will never have a capacity miss. How does this change our results? I'm only going to use LRU and Optimal to get the most important points across.

LRU

Direct

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 90
compulsory misses: 625
capacity misses: 0
conflict misses: 285
cache hit rate: 9.0%

Process returned 0 (0x0)   execution time : 0.141 s
Press any key to continue.
```

2-way

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 96
compulsory misses: 719
capacity misses: 0
conflict misses: 185
cache hit rate: 9.6%

Process returned 0 (0x0)   execution time : 0.105 s
Press any key to continue.
```

4-way

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 105
compulsory misses: 779
capacity misses: 0
conflict misses: 116
cache hit rate: 10.5%

Process returned 0 (0x0)   execution time : 0.107 s
Press any key to continue.
```

8-way

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 107
compulsory misses: 827
capacity misses: 0
conflict misses: 66
cache hit rate: 10.7%

Process returned 0 (0x0)   execution time : 0.105 s
Press any key to continue.
```

Fully

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 111
compulsory misses: 889
capacity misses: 0
conflict misses: 0
cache hit rate: 11.1%

Process returned 0 (0x0)   execution time : 0.129 s
Press any key to continue.
```

Optimal

Direct

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 90
compulsory misses: 625
capacity misses: 0
conflict misses: 285
cache hit rate: 9.0%

Process returned 0 (0x0)   execution time : 0.108 s
Press any key to continue.
```

2-way

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 111
compulsory misses: 719
capacity misses: 0
conflict misses: 170
cache hit rate: 11.1%

Process returned 0 (0x0)   execution time : 0.109 s
Press any key to continue.
```

4-way

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 111
compulsory misses: 779
capacity misses: 0
conflict misses: 110
cache hit rate: 11.1%

Process returned 0 (0x0)   execution time : 0.111 s
Press any key to continue.
```

8-way

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 111
compulsory misses: 827
capacity misses: 0
conflict misses: 62
cache hit rate: 11.1%

Process returned 0 (0x0)   execution time : 0.114 s
Press any key to continue.
```

Fully

```
================= Cache Statistics  =======================
for 1000 accesses ...
hits: 111
compulsory misses: 889
capacity misses: 0
conflict misses: 0
cache hit rate: 11.1%

Process returned 0 (0x0)   execution time : 0.131 s
Press any key to continue.
```

Doing this shows behavior more inline with what is expected, as well as drastically improving hit-rate performance overall (which is also expected from increasing cache size). Hit rate consistently got better as associativity scaled up and the optimal policy always produced the best possible hit rate (except for direct-mapped because there is nothing to 'optimize' for it). No capacity misses ever too of course.

Conclusions

Ultimately, this cache implementation supports direct-mapped (1 associative), n-associativities where n is a positive multiple of two from 2 to number of lines (2, 4, 8, … ,num_lines), fully associative either as n=0 or n=num_lines, and 3 replacement policies (rand, LRU, optimal). Increasing associativity reduced conflict misses, but higher associativity did not always improve hit rate for the given access patterns, until I changed the cache config to have more lines than accesses. The optimal policy performed best in all cases. These results highlight how cache performance depends strongly on not only cache size or associativity, but also workload behavior.